Longest Common Subsequence

Шерстобитов Андрей
332 группа

3 мая 2021 г.

Содержание

1 Задача с целыми числами **3**
1.1 Постановка задачи . . . 3
1.2 Введение в алгоритм . . . 3
1.3 Алгоритм . . . 4
1.4 Примеры . . . 4
1.5 Реализация алгоритма на C++ . . . 7

2 Задача с действительными числами **8**
2.1 Постановка задачи . . . 8
2.2 Введение в алгоритм . . . 9
2.3 Алгоритм . . . 9
2.3.1 Проверка алгоритма . . . 10
2.3.2 Возможные результаты . . . 10

1 Задача с целыми числами

1.1 Постановка задачи

Для данных последовательностей найти самую длинную общую подпоследовательность, при этом подпоследовательность может быть разрывна в данной последовательности.

Пример: Пусть даны две последовательности:

$$S1 = \{B, C, D, A, A, C, D\}$$
$$S2 = \{A, C, D, B, A, C\}$$

Тогда их общие подпоследовательности это

$$\{B, C\},\ \{C, D, A, C\},\ \{D, A, C\},\ \{A, A, C\},\ \{A, C\},\ \{C, D\},\ ...$$

Среди этих подпоследовательностей мы найдем самую длинную, а именно $\{C, D, A, C\}$

1.2 Введение в алгоритм

Прежде чем переходить непосредственно к алгоритму, рассмотрим несколько понятий:

Определение 1. *Назовем Ω множество элементов, из которых мы будем составлять наши последовательности. В данном разделе легче представлять это множество, как множество букв английского алфавита, а последовательности - соответственно - строки.*

Определение 2. *Знаком & будем обозначать стандартную конкатенацию последовательностей. Например: $\{a, b, c\}$ & $\{c, b, a\}$ $=$ $\{a, b, c, c, b, a\}$.*

Определение 3. *Префиксом последовательности $S = \{x_1, \ldots, x_n\}$ длины $0 < k \leq n$ обозначим $S_k = \{x_1, \ldots, x_k\}$*

Определение 4. *Назовем $\mathrm{LCS}(S1, S2)$ функцию, которая принимает на вход две последовательности $S1 = \{x_1, \ldots, x_n\}$ и $S2 = \{y_1, \ldots, y_n\}$ и возвращает наибольшую общую подпоследовательность C.*

Докажем вспомогательную лемму:

Лемма 1. *Функция $\mathrm{LCS}(S1, S2)$ обладает следующими свойствами:*

1. $\forall\ A \in \Omega$
$$\mathrm{LCS}(S1\ \&\ A,\ S2\ \&\ A) = \mathrm{LCS}(S1,\ S2)\ \&\ A$$

2. $\forall A \neq B \in \Omega$
$$\mathrm{LCS}(S1\ \&\ A, S2\ \&\ B) = \max\{\mathrm{LCS}(S1\ \&\ A, S2), \mathrm{LCS}(S1, S2\ \&\ B)\}$$

Доказательство. 1. Представим нахождение общей последовательности как последовательное сопоставление элементов последовательностей друг за другом без пересечений.

Рассмотрим $S1 = \{a, b, c, d, e, f\}$, $S2 = \{d, a, c, e, q\}$ и уже найденную для них общую подпоследовательность $C = \{a, c, e\}$, тогда

$$\begin{tikzcd}
\mathrm{a} \arrow[dr] & \mathrm{b} & \mathrm{c} \arrow[d] & \mathrm{d} & \mathrm{e} \arrow[dl] & \mathrm{f} \\
\mathrm{d} & \mathrm{a} & \mathrm{c} & \mathrm{e} & \mathrm{q} &
\end{tikzcd}$$

Добавив в конец каждой последовательности по одному элементу (пусть p), мы не нарушим наш алгоритм, так как новых пересечений быть не может.

$$\begin{array}{ccccccc} a & b & c & d & e & f & p \\ d & a & c & e & q & p & \end{array}$$

2. В противном случае, если последние элементы последовательностей $S1 = \{x_1, \ldots, x_n\}$, $S2 = \{y_1, \ldots, y_m\}$ различны ($x_n \neq y_m$), то какой-либо из них не входит в наибольшую <u>общую</u> подпоследовательность, иначе могут появиться самопересечения. Значит нужно рассмотреть последовательности $S1 = \{x_1, \ldots, x_n\}$, $S2' = \{y_1, \ldots, y_{m-1}\}$ и $S1' = \{x_1, \ldots, x_{n-1}\}$, $S2 = \{y_1, \ldots, y_m\}$ и среди них выбрать максимальную общую подпоследовательность.

□

Таким образом для последователностей $X = (x_1, \ldots, x_n), Y = (y_1, \ldots, y_m)$, их префиксов $X_{1,2,\ldots,n}, Y_{1,2,\ldots,m}$ наша функция $\mathrm{LCS}(X_i, Y_j)$ принимает вид:

$$\mathrm{LCS}(X_i, Y_j) = \begin{cases} \emptyset & \text{если } i = 0 \text{ или } j = 0 \\ \mathrm{LCS}(X_{i-1}, Y_{j-1}) \& x_i & \text{если } i, j > 0 \text{ и } x_i = y_j \\ \max\{\mathrm{LCS}(X_i, Y_{j-1}), \mathrm{LCS}(X_{i-1}, Y_j)\} & \text{если } i, j > 0 \text{ и } x_i \neq y_j \end{cases}$$

1.3 Алгоритм

Рассмотрим алгоритм для последовательностей $X = (x_1, \ldots, x_n)$, $Y = (y_1, \ldots, y_m)$:

1. Создаем таблицу размера $(n+1) \times (m+1)$, где первая строка и первый столбец заполнены нулями. Это нужно для случая, когда одна из последовательностей пустая.

2. Заполняем эту таблицу следующим образом:

   (a) Если элемент на текущей строке и текущей колонке совпадают, то по свойству 1 заполняем ячейку числом на 1 больше, чем в левой верхней диагональной ячейке. Проводим стрелку к этой же ячейке от текущей.

   (b) Иначе по свойству 2 берем максимальное значение с предыдущей колонки и предыдущей строки и заполняем текущую ячейку. Проводим стрелку к максимальному элементу. Если они равны, то к любому из них.

3. Повторяем предыдущий шаг до заполнения таблицы.

4. Получаем в правой нижней ячейке таблицы длину максимальной общей подпоследовательности.

5. Чтобы распечатать искомую подпоследовательность, начиинаем с правого нижнего элемента и двигаемся по стрелкам. Добравшись до первой строки или столбца получим нашу последовательность.

1.4 Примеры

Рассмотирм две последовательности $X = \{C, B, D, A\}, Y = \{A, C, A, D, B\}$ и выполним для них алгоритм.

1. Составляем требуемую таблицу.

	$\emptyset$	C	B	D	A
$\emptyset$	0	0	0	0	0
A	0				
C	0				
A	0				
D	0				
B	0				

2. Заполним первую строку соответственно нашему алгоритму и свойствам нашей функции LCS(S1, S2).

	$\emptyset$	C	B	D	A
$\emptyset$	0	0	0	0	0
A	0	0	0	0	1
C	0				
A	0				
D	0				
B	0				

3. Повторим для последующих строк.

	$\emptyset$	C	B	D	A
$\emptyset$	0	0	0	0	0
A	0	0	0	0	1
C	0	1	1	1	1
A	0	1	1	1	2
D	0	1	1	2	2
B	0	1	2	2	2

4. Значение в последней ячейке - длина наибольшей общей последовательности.

	$\emptyset$	C	B	D	A
$\emptyset$	0	0	0	0	0
A	0	0	0	0	1
C	0	1	1	1	1
A	0	1	1	1	2
D	0	1	1	2	2
B	0	1	2	2	2

5. Находим общие подпоследовательности, двигаясь по стрелочкам.

	$\emptyset$	C	B	D	A
$\emptyset$	0	0	0	0	0
A	0	0	0	0	1
C	0	1	1	1	1
A	0	1	1	1	2
D	0	1	1	2	2
B	0	1	2	2	2

Таким образом, получившиеся наибольшие общие подпоследовательности:

$$\{C, B\}, \{C, A\}, \{C, D\}$$

Замечание. *Данный пример показал, что наибольшая общая подпоследовательность не единственна.*

1.5 Реализация алгоритма на C++

```cpp
#include <vector>

template<typename T>
std::vector<T> LCS(const std::vector<T>& S1, const std::vector<T>& S2) {
    std::vector<std::vector<int>> table(S1.size() + 1,
                                        std::vector<int>(S2.size() + 1));
    size_t i, j;

    for (i = 0; i <= S1.size(); ++i) {
        for (j = 0; j <= S2.size(); ++j) {
            if (i == 0 || j == 0)
                table[i][j] = 0;
            else if (S1[i - 1] == S2[j - 1])
                table[i][j] = table[i - 1][j - 1] + 1;
            else
                table[i][j] = std::max(table[i - 1][j], table[i][j - 1]);
        }
    }

    int index = table[S1.size()][S2.size()];
    std::vector<T> lcs(index);
    i = S1.size(), j = S2.size();
    while (i > 0 && j > 0) {
        if (S1[i - 1] == S2[j - 1]) {
          lcs[index - 1] = S1[i - 1];
          i--, j--, index--;
        } else if (table[i - 1][j] > table[i][j - 1]) {
            i--;
        } else {
            j--;
        }
    }

    return lcs;
}
```

Сложность работы: $\mathcal{O}(mn)$, где m и n - длины последовательностей.
Сложнсоть по памяти: $\mathcal{O}(mn)$, где m и n - длины последовательностей.

2 Задача с действительными числами

Задачу нахождения общей подпоследовательности для двух последовательностей S^1, S^2 с плавающей точкой можно поставить по-разному.

Самый наивный вариант это получить от пользователя некую погрешность ε, с помощью которой мы должны будем определить является ли каждый i-ый элемент членом общей подпоследовательности или нет.

Пытаясь решить эту задачу аналогично задаче с целыми числами мы столкнемся с несколькими трудностями:

1. Нужно определиться как работать с данной погрешностью, ведь просто напросто сравнивать разницу i-ого и j-ого элементов последоватностей с ε и строить таблицу аналогично первой задаче не получится.

   Например: S^1 и S^2 такие, что $\|S^1\|_2 < \varepsilon$ и $\|S^2\|_2 < \varepsilon$. Что это значит для нас в таком случае? Это значит, что эти две последовательности будут совпадать, так как $\forall i, j\ |S^1_i - S^2_j| < \varepsilon$, что не будет правильным ответом.

2. Пусть мы придумали какой-то функционал $\phi(S^1_i, S^2_j)$, который позволит нам как-то правильно заполнить таблицу. Возникает следующая проблема. Как с помощью это функционала понимать какой элемент является очередным элементом искомой подпоследовательности?

Посмотрим на нашу задачу под другим углом:

Давайте предположим, что одна из последовательностей уже является наибольшей общей подпоследовательностью. Остается придумать как понять верна ли наша гипотеза. Попытаемся формализовать такую постановку.

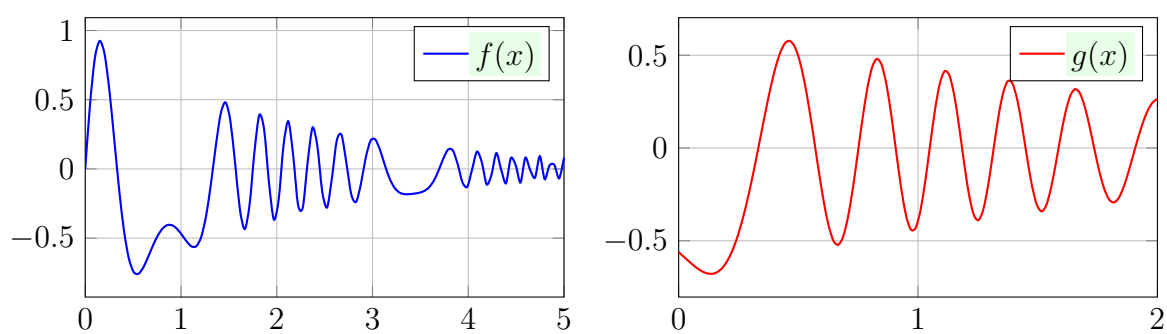
2.1 Постановка задачи

Для двух заданных последовательностей понять, можно ли сопоставить одну последовательность с подмножеством значений другой последовательности? Дать какое-либо определение сопоставлению и показать качество получившегося сопоставления.

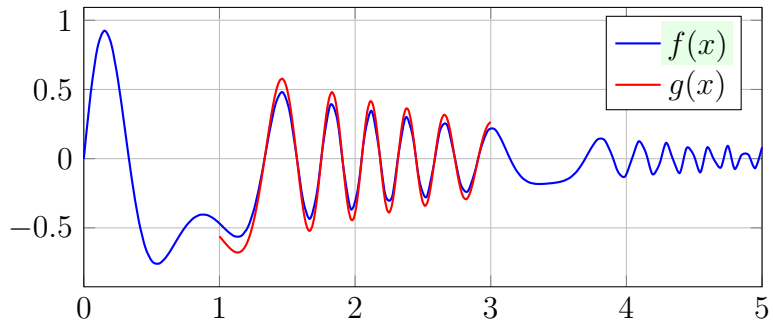
Пояснительный примеры:

1. Интерпретируем задачу через непрерывные функции:
   Рассмотрим две непрерывные функции одной переменной $f(x)$ на отрезке $[a, b]$ и $g(x)$ на отрезке $[c, d]$.
   Вопрос: Можно ли утверждать, что значения $g(x)$ на отрезке $[c, d]$ являются подмножеством значений функции $f(x)$ на отрезке $I \subset [a, b]$?

   Возьмем две функции: $f(x)$ на отрезке $[0, 5]$ и $g(x)$ на отрезке $[0, 2]$.



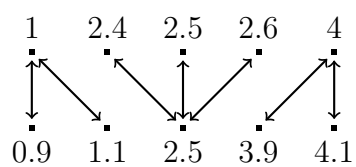
   Тогда наилучшим соответствим между f и g будет:



2. (a) Даны две последовательности

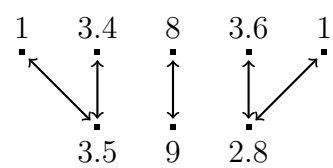
$$S1 = \{1, 2.4, 2.5, 2.6, 4\},\ S2 = \{0.9, 1.1, 2.5, 3.9, 4.1\}$$

Тогда их наилучшим сопоставлением будет отображение:



(b)

$$S1 = \{1,\ 3.4,\ 8,\ 3.6,\ 1\},\ S2 = \{3.5,\ 9,\ 2.8\}$$



2.2 Введение в алгоритм

Мы будем искать наилучшее сопоставление используя динамическое программирование аналогично и первой задаче с целыми числами.

В качестве функционала качества рассмотрим функцию

$$M = \frac{1}{k}\sum_{i=1}^{k}(a_i - b_{j(i)})^2 \xrightarrow[\text{задача}]{} \min$$

где k - количество сопоставлений в нашем алгоритме.

Вся задача теперь сводится к тому, чтобы отобразить одну последовательность в другую так, чтобы функционал качества был минимальным.

2.3 Алгоритм

Рассмотрим алгоритм для последовательностей $X = (x_1, \ldots, x_n)$, $Y = (y_1, \ldots, y_m)$.

1. Создаем таблицу размера $n \times m$.

2. Заполняем эту таблицу следующим образом:

2.3.1 Проверка алгоритма

2.3.2 Возможные результаты